

DR1 – TP3

Programação Orientada a Objetos em Java

Aluno: Gabriel Alves Sandre da Silva

Este trabalho apresenta as soluções dos doze exercícios propostos, abordando classes, objetos, atributos, métodos, getters e setters, construtores e métodos de cálculo. Todo o código foi escrito em Java, compilado com javac e executado com java; as saídas mostradas em cada exercício são as saídas reais obtidas na execução.

Exercício 1 – Conceitos de Classe, Objeto, Campos e Métodos

Classe é o molde que define quais características e comportamentos os objetos daquele tipo terão. Ela não guarda dados por si só: descreve como os objetos serão. **Objeto** é uma instância criada a partir desse molde, com valores próprios; cada objeto ocupa seu próprio espaço na memória. **Campos** (ou atributos) são as variáveis declaradas na classe, responsáveis por armazenar os dados de cada objeto. **Métodos** são os comportamentos que o objeto pode executar, podendo ler ou alterar os seus próprios atributos.

No exemplo abaixo, Carro é a classe; marca e modelo são os campos; ligar() é o método, que utiliza os dois campos na mensagem exibida; e meuCarro é o objeto criado com new Carro().

Arquivo: Carro.java

```
// Exercício 1 - Exemplo de classe, objeto, campos e métodos.
public class Carro {
    String marca;
    String modelo;

    void ligar() {
        System.out.println("O carro " + marca + " " + modelo + " está ligado.");
    }
}
```

Arquivo: Main.java

```
// Exercício 1 - Criação de um objeto a partir da classe Carro.
public class Main {
    public static void main(String[] args) {
        Carro meuCarro = new Carro();
        meuCarro.marca = "Toyota";
        meuCarro.modelo = "Corolla";
        meuCarro.ligar();
    }
}
```

Saída no console (java Main):

```
O carro Toyota Corolla está ligado.
```

Exercício 2 – Criando a Classe “Produto”

A classe Produto representa cada item cadastrado no supermercado. O atributo **nome** (String) identifica o produto na listagem e nas buscas do sistema. O atributo **preço** (double) registra o valor de venda e é do tipo decimal porque preços raramente são números inteiros. O atributo

quantidadeEmEstoque (int) informa quantas unidades existem, sendo inteiro porque o controle é feito por unidades e é ele que permite saber quando repor a mercadoria.

Estrutura inicial da classe (versão do exercício 2)

```
public class Produto {  
    String nome;  
    double preco;  
    int quantidadeEmEstoque;  
}
```

Exercício 3 – Métodos Básicos da Classe “Produto”

O método `alterarPreco` recebe um novo valor e o grava no atributo `preco`, sendo usado em uma remarcação. `alterarQuantidade` faz o mesmo com `quantidadeEmEstoque`, refletindo entradas e saídas de mercadoria. Já `exibirInformacoes` não altera nada: apenas lê os três atributos e mostra no console o estado atual do objeto, servindo como o registro pedido no contexto.

Métodos acrescentados à classe `Produto`

```
void alterarPreco(double novoPreco) {  
    preco = novoPreco;  
}  
  
void alterarQuantidade(int novaQuantidade) {  
    quantidadeEmEstoque = novaQuantidade;  
}  
  
void exibirInformacoes() {  
    System.out.println("Nome: " + nome);  
    System.out.println("Preço: " + preco);  
    System.out.println("Quantidade em estoque: " + quantidadeEmEstoque);  
}
```

Exercício 4 – Testando a Classe “Produto”

A classe `AppProduto` contém o método `main`. Nele o objeto é instanciado, recebe os valores iniciais (Arroz, 25.00 e 10 unidades), tem preço e quantidade atualizados por `alterarPreco` e `alterarQuantidade` e, por fim, exibe as informações — confirmando que as alterações foram aplicadas.

Trecho do `main` de `AppProduto.java`

```
Produto produto = new Produto();  
produto.nome = "Arroz";  
produto.preco = 25.00;  
produto.quantidadeEmEstoque = 10;  
  
produto.alterarPreco(27.50);  
produto.alterarQuantidade(15);  
produto.exibirInformacoes();
```

Saída correspondente no console:

```
Nome: Arroz  
Preço: 27.5  
Quantidade em estoque: 15
```

Exercício 5 – Criando Métodos de Propriedade (Getters e Setters)

Os **getters** devolvem o valor de cada atributo e os **setters** recebem um novo valor e o gravam. Mesmo sem usar modificadores de visibilidade, eles são úteis porque centralizam o acesso aos dados: se um dia for preciso validar o preço (impedir valor negativo, por exemplo) ou registrar quem alterou o estoque, basta mudar o setter, e todo o restante do sistema continua funcionando sem alteração. Também padronizam o código, já que todo o programa passa a ler e escrever os atributos da mesma maneira.

Getters e setters da classe Produto

```
String getNome() {
    return nome;
}

double getPreco() {
    return preco;
}

int getQuantidadeEmEstoque() {
    return quantidadeEmEstoque;
}

void setNome(String novoNome) {
    nome = novoNome;
}

void setPreco(double novoPreco) {
    preco = novoPreco;
}

void setQuantidadeEmEstoque(int novaQuantidade) {
    quantidadeEmEstoque = novaQuantidade;
}
```

Exemplo de uso no main

```
Produto outroProduto = new Produto();
outroProduto.setNome("Feijão");
outroProduto.setPreco(3.75);
outroProduto.setQuantidadeEmEstoque(20);

int quantidade = outroProduto.getQuantidadeEmEstoque();
System.out.println("Nome: " + outroProduto.getNome());
System.out.println("Preço: " + outroProduto.getPreco());
System.out.println("Quantidade em estoque: " + quantidade);
```

Saída correspondente no console:

```
Nome: Feijão
Preço: 3.75
Quantidade em estoque: 20
```

A chamada `setPreco(3.75)` seguida de `getPreco()` confirma que o valor foi atualizado.

Exercício 6 – Adicionando Construtores à Classe “Produto”

O construtor é executado no momento em que o objeto é criado com `new`. Ele recebe nome, preço e quantidade como parâmetros e usa `this` para diferenciar o atributo do parâmetro de mesmo nome, atribuindo os valores de uma só vez. Assim o objeto já nasce completo e válido, em vez de existir por um tempo com atributos vazios até que os setters sejam chamados um a um. Isso reduz a chance de esquecer algum dado e deixa o código de criação mais curto e legível.

Foi mantido também um construtor sem parâmetros, para que as criações dos exercícios 4 e 5 (`new Produto()`) continuem válidas na mesma classe.

Construtores da classe **Produto**

```
Produto() {  
}  
  
Produto(String nome, double preco, int quantidadeEmEstoque) {  
    this.nome = nome;  
    this.preco = preco;  
    this.quantidadeEmEstoque = quantidadeEmEstoque;  
}
```

Criação do objeto usando o construtor

```
Produto leite = new Produto("Leite", 5.50, 30);  
leite.exibirInformacoes();
```

Saída correspondente no console:

```
Nome: Leite  
Preço: 5.5  
Quantidade em estoque: 30
```

A classe **Produto** completa — reunindo os atributos do exercício 2, os métodos do exercício 3, os getters e setters do exercício 5 e os construtores do exercício 6 — está no anexo, ao final deste documento, junto da classe de teste **AppProduto** e da saída completa da execução.

Exercício 7 – Modelando uma Conta Bancária

A classe **Conta** segue as convenções do Java (nome no singular e iniciando com letra maiúscula) e reúne os cinco atributos pedidos: **titular** (dono da conta), **numero** (identificador da conta), **agencia** (mantida como `String` por poder começar com zero, como em “0001”), **saldo** (valor disponível, decimal) e **dataAbertura** (registro de quando a conta foi criada).

Estrutura inicial da classe (versão do exercício 7)

```
public class Conta {  
    String titular;  
    int numero;  
    String agencia;  
    double saldo;  
    String dataAbertura;  
}
```

Exercício 8 – Criando Métodos

`saca(double valor)` subtrai o valor informado do saldo. `deposita(double valor)` soma o valor ao saldo. `calculaRendimento()` não recebe parâmetro e apenas devolve o saldo multiplicado por 0.1, ou seja, 10% — repare que ele retorna o valor em vez de alterar o saldo, exatamente como pede o enunciado.

Arquivo: Conta.java

```
// Exercícios 7 e 8 - Classe Conta do sistema bancário.
public class Conta {

    // Exercício 7 - Atributos.
    String titular;
    int numero;
    String agencia;
    double saldo;
    String dataAbertura;

    // Exercício 8 - Métodos de saque, depósito e rendimento.
    void saca(double valor) {
        saldo = saldo - valor;
    }

    void deposita(double valor) {
        saldo = saldo + valor;
    }

    double calculaRendimento() {
        return saldo * 0.1;
    }
}
```

Exercício 9 – Vamos testar nossa classe

A classe TestaConta instancia uma conta, preenche os cinco atributos e executa as três operações, exibindo o saldo antes e depois de cada uma. Partindo de 1000.00, o saque de 200.00 leva o saldo a 800.00, o depósito de 500.00 o leva a 1300.00 e o rendimento devolve 130.00, que corresponde a 10% desse saldo final — confirmando que os métodos funcionam como esperado.

Arquivo: TestaConta.java

```
// Exercício 9 - Classe de teste da Conta.
public class TestaConta {
    public static void main(String[] args) {
        Conta conta = new Conta();
        conta.titular = "Gabriel Alves Sandre da Silva";
        conta.numero = 1234;
        conta.agencia = "0001";
        conta.saldo = 1000.00;
        conta.dataAbertura = "08/09/2026";

        System.out.println("Titular: " + conta.titular);
        System.out.println("Conta: " + conta.numero + " / Agência: " + conta.agencia);
        System.out.println("Data de abertura: " + conta.dataAbertura);
        System.out.println("Saldo inicial: " + conta.saldo);

        conta.saca(200.00);
        System.out.println("Saldo após saque de 200.00: " + conta.saldo);

        conta.deposita(500.00);
        System.out.println("Saldo após depósito de 500.00: " + conta.saldo);

        double rendimento = conta.calculaRendimento();
        System.out.println("Rendimento (10% do saldo): " + rendimento);
    }
}
```

Saída no console (java TestaConta):

```
Titular: Gabriel Alves Sandre da Silva
Conta: 1234 / Agência: 0001
Data de abertura: 08/09/2026
Saldo inicial: 1000.0
Saldo após saque de 200.00: 800.0
Saldo após depósito de 500.00: 1300.0
Rendimento (10% do saldo): 130.0
```

Exercício 10 – Definindo Classes para Formas Geométricas

As classes `Circulo` e `Esfera` foram criadas de forma independente, sem herança ou qualquer recurso avançado. O atributo **raio** é fundamental em ambas porque é a única medida de que os cálculos dependem: a área do círculo é obtida a partir do raio ao quadrado e o volume da esfera, do raio ao cubo. Sem esse atributo, não haveria como calcular nem uma coisa nem outra. Ele é `double` por aceitar medidas fracionadas.

Estrutura inicial das classes (versão do exercício 10)

```
public class Circulo {
    double raio;
}

public class Esfera {
    double raio;
}
```

Exercício 11 – Criando Métodos de Cálculo

`calcularArea()` retorna $\pi \times \text{raio}^2$ e `calcularVolume()` retorna $(4/3) \times \pi \times \text{raio}^3$. Foi utilizado `Math.PI`, constante já fornecida pelo Java, que é mais precisa do que digitar o valor aproximado. A divisão foi escrita como `4.0 / 3.0` justamente para que o Java faça a conta com decimais: se fosse `4 / 3`, o resultado seria 1 e o volume sairia errado. Os dois métodos apenas devolvem o resultado, sem imprimir nada.

Arquivo: `Circulo.java`

```
// Exercícios 10 e 11 - Figura 2D.
public class Circulo {
    double raio;

    double calcularArea() {
        return Math.PI * (raio * raio);
    }
}
```

Arquivo: `Esfera.java`

```
// Exercícios 10 e 11 - Figura 3D.
public class Esfera {
    double raio;

    double calcularVolume() {
        return (4.0 / 3.0) * Math.PI * (raio * raio * raio);
    }
}
```

Exercício 12 – Testando as Classes de Figuras

A classe TestaFiguras instancia um círculo de raio 3.0 e uma esfera de raio 5.0, chama os métodos de cálculo e exibe os resultados. Os valores obtidos conferem com o cálculo manual: $\pi \times 9 \approx 28,27$ e $(4/3) \times \pi \times 125 \approx 523,60$.

Arquivo: TestaFiguras.java

```
// Exercício 12 - Classe de teste das figuras geométricas.
public class TestaFiguras {
    public static void main(String[] args) {
        Circulo circulo = new Circulo();
        Esfera esfera = new Esfera();

        circulo.raio = 3.0;
        esfera.raio = 5.0;

        System.out.println("Área do círculo (raio 3.0): " + circulo.calcularArea());
        System.out.println("Volume da esfera (raio 5.0): " + esfera.calcularVolume());
    }
}
```

Saída no console (java TestaFiguras):

```
Área do círculo (raio 3.0): 28.274333882308138
Volume da esfera (raio 5.0): 523.5987755982989
```

Anexo – Classe Produto completa (exercícios 2 a 6)

Como a classe Produto foi construída em etapas ao longo dos exercícios 2, 3, 5 e 6, segue abaixo o arquivo final reunindo tudo, com a classe de teste e a saída completa da execução.

Arquivo: Produto.java (parte 1 de 2)

```
// Exercícios 2, 3, 5 e 6 - Classe Produto do sistema do supermercado.
public class Produto {

    // Exercício 2 - Atributos.
    String nome;
    double preco;
    int quantidadeEmEstoque;

    // Exercício 6 - Construtor sem parâmetros (permite criar o objeto e
    // preencher os atributos depois, como nos exercícios 4 e 5).
    Produto() {
    }

    // Exercício 6 - Construtor que já inicializa os três atributos.
    Produto(String nome, double preco, int quantidadeEmEstoque) {
        this.nome = nome;
        this.preco = preco;
        this.quantidadeEmEstoque = quantidadeEmEstoque;
    }

    // Exercício 3 - Métodos básicos de atualização e exibição.
    void alterarPreco(double novoPreco) {
        preco = novoPreco;
    }

    void alterarQuantidade(int novaQuantidade) {
        quantidadeEmEstoque = novaQuantidade;
    }

    void exibirInformacoes() {
        System.out.println("Nome: " + nome);
        System.out.println("Preço: " + preco);
        System.out.println("Quantidade em estoque: " + quantidadeEmEstoque);
    }
}
```

Arquivo: Produto.java (parte 2 de 2)

```
// Exercício 5 - Getters.
String getNome() {
    return nome;
}

double getPreco() {
    return preco;
}

int getQuantidadeEmEstoque() {
    return quantidadeEmEstoque;
}

// Exercício 5 - Setters.
void setNome(String novoNome) {
    nome = novoNome;
}

void setPreco(double novoPreco) {
    preco = novoPreco;
}

void setQuantidadeEmEstoque(int novaQuantidade) {
    quantidadeEmEstoque = novaQuantidade;
}
}
```

Arquivo: AppProduto.java

```
// Exercícios 4, 5 e 6 - Testes da classe Produto.
public class AppProduto {
    public static void main(String[] args) {

        // Exercício 4 - Instanciação e uso dos métodos de atualização.
        Produto produto = new Produto();
        produto.nome = "Arroz";
        produto.preco = 25.00;
        produto.quantidadeEmEstoque = 10;

        produto.alterarPreco(27.50);
        produto.alterarQuantidade(15);
        produto.exibirInformacoes();

        // Exercício 5 - Uso dos getters e setters.
        System.out.println();
        Produto outroProduto = new Produto();
        outroProduto.setNome("Feijão");
        outroProduto.setPreco(3.75);
        outroProduto.setQuantidadeEmEstoque(20);

        int quantidade = outroProduto.getQuantidadeEmEstoque();
        System.out.println("Nome: " + outroProduto.getNome());
        System.out.println("Preço: " + outroProduto.getPreco());
        System.out.println("Quantidade em estoque: " + quantidade);

        // Exercício 6 - Criação do objeto usando o construtor com parâmetros.
        System.out.println();
        Produto leite = new Produto("Leite", 5.50, 30);
        leite.exibirInformacoes();
    }
}
```

Saída no console (java AppProduto):

```
Nome: Arroz
Preço: 27.5
Quantidade em estoque: 15

Nome: Feijão
Preço: 3.75
Quantidade em estoque: 20

Nome: Leite
Preço: 5.5
Quantidade em estoque: 30
```

Organização dos arquivos e execução

Cada classe pública ficou em seu próprio arquivo, com o mesmo nome da classe, como exige o Java: Carro.java, Main.java, Produto.java, AppProduto.java, Conta.java, TestaConta.java, Circulo.java, Esfera.java e TestaFiguras.java. Para compilar e executar, basta compilar os arquivos em conjunto e rodar a classe que contém o método main desejado:

```
javac -encoding UTF-8 -d out src/*.java

java -cp out Main           # Exercício 1
java -cp out AppProduto     # Exercícios 4, 5 e 6
java -cp out TestaConta     # Exercício 9
java -cp out TestaFiguras   # Exercício 12
```